Regular Expressions in Python

Lab Worksheet: Matching, Search & Replace, and Patterns

Module: Python Programming | Topic: re module | Estimated time: 90 minutes

Learning Objectives

- Use Python's `re` module to match strings against patterns.
- Distinguish between `match()`, `search()`, `fullmatch()`, `findall()`, and `finditer()`.
- Perform search-and-replace operations with `sub()` and `subn()`.
- Build patterns using character classes, quantifiers, anchors, and groups.
- Apply regular expressions to realistic text-processing problems (validation, extraction, cleaning).

How This Lab Works

This worksheet contains six tasks of increasing difficulty. Each task states an objective, gives background notes and a worked example, and then asks you to write your own code in a new Python file (or Jupyter cell). Run every example yourself before attempting the exercise — do not just read the code. Keep a single script called `regex_lab.py` and add each task's solution under a comment heading (e.g. `# Task 1`).

Setup

The `re` module is part of the Python standard library, so no installation is required. Start every script with:

```
import re
```

Total tasks	Difficulty	Estimated time	Deliverable
6	Beginner → Intermediate	90 minutes	regex_lab.py

TASK 1 | Basic Pattern Matching — 10 pts

Objective: Learn the difference between `re.match()`, `re.search()`, and `re.fullmatch()`.

Background

`re.match(pattern, string)` checks for a match only at the *beginning* of the string. **`re.search(pattern, string)`** scans the whole string and returns the first match found anywhere. **`re.fullmatch(pattern, string)`** succeeds only if the *entire* string matches the pattern. All three return a `Match` object on success, or `None` on failure.

Worked Example

```
import re

text = "The quick brown fox jumps"

m1 = re.match(r"The", text)      # matches: starts at position 0
m2 = re.match(r"quick", text)    # None: "quick" is not at the start
m3 = re.search(r"quick", text)   # matches: found later in the string
m4 = re.fullmatch(r"The.*fox.*jumps", text)  # matches: covers whole string

print(m1.group())   # "The"
print(m3.span())    # (4, 9) -> start/end indices of "quick"
```

Your Turn

- Given `sentence = "1024 requests were served in 3 seconds"`, use `re.match()` to test whether the sentence starts with a digit.
- Use `re.search()` to find the word `"served"` anywhere in the sentence and print its start/end position with `.span()`.
- Use `re.fullmatch()` to check whether the string `"12345"` consists *only* of digits, then test it again on `"123a5"`.
- In one or two sentences, explain in a comment why `match()` and `fullmatch()` gave different results for the same pattern on a string that starts with digits but contains other characters later.

TASK 2 | Finding All Matches — 10 pts

Objective: Extract every occurrence of a pattern using `re.findall()` and `re.finditer()`.

Background

`re.findall(pattern, string)` returns a list of all non-overlapping matches as strings (or tuples, if the pattern has groups). **`re.finditer(pattern, string)`** returns an iterator of `Match` objects, which is more memory-efficient and gives you access to positions via `.span()`.

Worked Example

```
import re

log = "Errors on lines 12, 45 and 108; warnings on lines 7 and 91"

numbers = re.findall(r"\d+", log)
print(numbers)   # ['12', '45', '108', '7', '91']

for match in re.finditer(r"\d+", log):
    print(match.group(), match.span())
```

Your Turn

- Given a paragraph of text, use `re.findall()` to extract every word that is written in all capital letters (e.g. "NASA", "USA").
- Use `re.finditer()` on the same paragraph to find every word longer than 6 characters, and print each match together with its start index.
- Given `prices = "apples: $3.50, bananas: $1.20, mango: $4.75"`, extract all the dollar amounts (including the decimal part) as a list of strings.
- Count how many times the pattern occurs using `len()` on the result of `findall()`.

TASK 3 | Search and Replace — 15 pts

Objective: Modify text programmatically with `re.sub()` and `re.subn()`, including replacement functions and back-references.

Background

`re.sub(pattern, repl, string, count=0)` replaces matches of `pattern` with `repl`, which may be a string (optionally using `\1`, `\2` back-references to captured groups) or a function that receives the `Match` object and returns the replacement text. **`re.subn()`** works the same way but also returns the number of substitutions made, as a tuple.

Worked Example

```
import re

text = "Contact us at 555-123-4567 or 555-987-6543"

# 1. Simple replacement
masked = re.sub(r"\d{3}-\d{3}-\d{4}", "[PHONE]", text)

# 2. Replacement using back-references (swap day/month in a date)
date_fixed = re.sub(r"(\d{2})/(\d{2})/(\d{4})", r"\2/\1/\3", "31/01/2024")

# 3. Replacement using a function
def to_upper(match):
    return match.group().upper()

shouted = re.sub(r"\bpython\b", to_upper, "I love python and python loves me")

# 4. subn() also reports how many replacements were made
result, n = re.subn(r"\s+", " ", "too    many     spaces")
print(result, n)   # 'too many spaces' 2
```

Your Turn

- Write a function that redacts every email address in a block of text, replacing it with `"[EMAIL HIDDEN]"`, using `re.sub()`.
- Given a string of names in the form `"Doe, John"`, use groups and back-references in `re.sub()` to convert it to `"John Doe"`.
- Write a replacement function that finds every number in a sentence and replaces it with that number doubled (e.g. `"I have 3 apples"` → `"I have 6 apples"`). Hint: convert `match.group()` to `int` inside the function.
- Use `re.subn()` to collapse repeated punctuation (e.g. `"Wait!!!"` → `"Wait!"`) in a sample string and print how many replacements were made.

TASK 4 | Building Patterns — 20 pts

Objective: Practice the core building blocks of regex syntax: character classes, quantifiers, anchors, alternation, and groups.

Reference Table

Symbol	Meaning	Example
`.`	Any character except newline	a.c matches "abc", "axc"
`\d  \D`	Digit / non-digit	\d+ matches "2024"
`\w  \W`	Word char / non-word char	\w+ matches "hello_1"
`\s  \S`	Whitespace / non-whitespace	\s+ matches " "
`[...]`	Character class (set)	[aeiou] matches any vowel
`[^...]`	Negated character class	[^0-9] matches any non-digit
`*  +  ?`	0+, 1+, 0-or-1 repetitions	ab* , ab+ , ab?
`{m,n}`	Between m and n repetitions	\d{2,4} matches 2 to 4 digits
`^  $`	Start / end of string (or line)	^Hello matches at position 0
`\b`	Word boundary	\bcat\b matches whole word "cat"
`( )`	Capturing group	(\d{3})-(\d{4})
`(?: )`	Non-capturing group	(?:abc)+
`|`	Alternation (OR)	cat|dog matches "cat" or "dog"

Worked Example

```
import re

# A pattern with named groups, quantifiers, and alternation
pattern = r"(?P<code>US|UK|IN)-(?P<id>\d{4,6})"

m = re.search(pattern, "Shipment ref: IN-48213 arrived")
print(m.group("code"))  # "IN"
print(m.group("id"))    # "48213"
```

Your Turn

- Write a pattern that matches a valid Python variable name (starts with a letter or underscore, followed by any number of letters, digits, or underscores) and test it against `"_count2"`, `"2fast"`, and `"total_sum"`.
- Write a pattern using alternation that matches either `"cat"`, `"dog"`, or `"bird"` as a whole word, and use it with `findall()` on a sentence containing pets.
- Write a pattern with a quantifier range `{m,n}` that matches hexadecimal color codes such as `"#FFAA00"` or `"#000"` (3 or 6 hex digits after the hash).

- Use named groups (`(?P<name>...)`) to parse a log line of the form `"2024-06-01 08:15:32 ERROR Disk full"` into date, time, level, and message parts, then print each part using `match.group("name")`.

TASK 5 | Practical Applications 25 pts

Objective: Combine everything you have learned to solve realistic text-processing problems.

5.1 — Email Validator

Write a function `is_valid_email(s)` that returns `True` if `s` looks like a valid email address (one or more word characters / dots, an `@`, a domain name, a dot, and a 2-6 letter top-level domain). Test it on at least 4 valid and 4 invalid examples, including edge cases like `"a@b.c"` and `"no-at-sign.com"`.

5.2 — Phone Number Extractor

Given a block of text containing phone numbers in mixed formats (`555-123-4567`, `(555) 123-4567`, `555.123.4567`), write a single pattern that extracts all of them regardless of format, and normalize each result to `555-123-4567` style using `re.sub()`.

5.3 — Date Extraction and Reformatting

Given text containing dates in `DD/MM/YYYY` format, extract all of them with `re.findall()` using groups, then use `re.sub()` with back-references to rewrite every date into `YYYY-MM-DD` (ISO) format.

5.4 — Whitespace and HTML Cleanup

Write a function `clean_text(html)` that: (a) strips out all HTML tags such as <b> or </p>, and (b) collapses any run of multiple spaces, tabs, or newlines into a single space, returning the trimmed result.

5.5 — Password Strength Checker

Write a function `check_password(pw)` that uses several regex tests (not one giant pattern) to verify a password is at least 8 characters long and contains at least one uppercase letter, one lowercase letter, one digit, and one symbol from `!@#$%^&*`. Have the function return a list of the specific rules that failed, so the caller gets useful feedback.

TASK 6 | Challenge — Mini Log Parser 20 pts

Objective: Bring matching, groups, search-and-replace, and iteration together in one small program.

Scenario

You are given a multi-line server log stored in a string, where each line looks like:

```
[2024-06-01 08:15:32] ERROR  user=jsmith  msg="Disk quota exceeded"
[2024-06-01 08:16:05] INFO   user=agarcia msg="Login successful"
[2024-06-01 08:17:44] WARN   user=jsmith  msg="High memory usage"
```

Requirements

- Write one regex pattern with named groups that captures the `timestamp`, `level`, `user`, and `msg` from each line.
- Use `re.finditer()` to loop over all lines and build a list of dictionaries, one per log entry, using `match.groupdict()`.
- Print a summary count of how many `ERROR`, `WARN`, and `INFO` entries exist.
- Use `re.sub()` to produce a redacted version of the log where every `user=<name>` is replaced with `user=<hidden>`, without touching anything else on the line.
- **Bonus:** Sort the parsed entries by user name and print only the `ERROR` entries for each user.

Submission Checklist

- A single file `regex_lab.py` with each task's code under a clearly labelled comment heading (`# Task 1`, `# Task 2`, ...).
- Every exercise includes at least one `print()` statement showing the result, so your output is visible when the script is run.
- Code runs without errors on Python 3.8 or later using only the standard library.
- Short comments explaining any pattern that isn't immediately obvious.

Tip: while designing a pattern, test it interactively (e.g. in a Python shell or a tool such as regex101.com) before pasting it into your script — this makes it much faster to spot mistakes in character classes and escaping.